

III. РЕАЛИЗАЦИЯ НА ЗАДАЧИ С УПРАВЛЯВАЩИ ОПЕРАТОРИ

Цел на упражнението

Това упражнение разглежда предимно задачи, свързани с основни управляващи оператори, създаване на игри, dll файлове и представя спецификата на контролите DateTimePicker и Timer.

1. Линеини алгоритми

Линейни алгоритми са тези, при които последователността от действия е една и съща за всички входни данни. Всяка стъпка на алгоритъма има само една предходна и една следваща.

2. Разклонени алгоритми

Разклонени алгоритми са тези, при които се прави избор между няколко възможни условия.

2.1. Логически изрази

Таблица 3.1.

Логически оператор	Стилизиран запис	Наименование
&&	and	и
	or	или
!	not	логическо отрицание
^	xor	изключващо или

Таблица 3.2.

x	y	!x	x && y	x y	x ^ y
true	true	false	true	true	false
true	false	false	false	true	true
false	true	true	false	true	true
false	false	true	false	false	false

2.2. Оператор if

2.2.1. if

```
if(condition)
{
    // code block to be executed when if condition evaluates to true
}
```

2.2.2. if-else

```
if(condition)
{
    // code block to be executed when if condition evaluates to true
}
else if(condition)
{
    // code block to be executed when if condition evaluates to false
}
```

2.2.3. Каскадно подреждане на оператори if

```
if (condition-expression-1)
{
    // statements executed if boolean-expression-1 is true
}
else if (boolean-expression-2)
{
    // statements executed if boolean-expression-2 is true
}
else if (boolean-expression-3)
{
    // statements executed if boolean-expression-3 is true
}
else
{
}
```

```

    // statements executed if all above expressions are false
}

```

2.2.4. Вложени оператори if

```

if (boolean-expression)
{
    if (nested-expression-1)
    {
        // code to be executed
    }
    else
    {
        // code to be executed
    }
}
else
{
    if (nested-expression-2)
    {
        // code to be executed
    }
    else
    {
        // code to be executed
    }
}

```

2.2.5. Тернарен оператор ?

Условният оператор ? : е оператор в езика C и C-подобните езици. Известен е още като тернарен оператор, тъй като е единственият оператор, който приема 3 операнда.

операнд1 ? операнд2 : операнд3

Първият операнд или условието на условната конструкция може да бъде булева променлива или булев израз и може да приема двете булеви стойности true (истина) и false (лъжа). Ако след извършването на необходимите преобразувания операнд1 се сведе до истинно твърдение (true), то тогава след изпълнението си тернарният оператор ще върне стойността на операнд2, в противен случай (false), върнатата стойност ще бъде стойността на операнд3.

2.3. Оператор за избор на варианти за switch

```

switch (testValue) {
case (value1): statements1;
break;
case (value2): statements2;
break;
default: statementsDefault;
break; }

```

3. Оператор за цикъл

3.1. Цикъл for

```

for (initialization; doneTest; next)
{
    statements..... }

```

initialization – подготвя цикъла за стартиране. Тази част декларира и инициализира променлива на цикъла.

doneTest – представлява булев израз, който определя кога да бъде спрян цикълът, който продължава докато изразът е верен.

next – подготвя цикъла за следваща стъпка, т.е. увеличава променливата на цикъла, която е декларирана в initialization.

statements – оператори, които ще бъдат изпълнени в цикъла.

3.2. Цикъл foreach

Цикълът foreach изпълнява оператор или блок от изрази за всеки елемент в екземпляр от типа, който реализира интерфейса IEnumerable или Generic.IEnumerable<T>.

```

foreach (variableDeclaration in items)
{
    statements..... }

```

variableDeclaration – декларира променливата за цикъла. Нейният тип трябва да бъде същият като елементите в масива или колекцията.

items – масив или колекция от елементи, през които ще премине цикълът

statements – оператори, които ще се изпълнят от цикъла

3.3. Цикли с условия. Итерационни алгоритми

3.2.1. Цикъл while

Това е цикъл, който се изпълнява, докато условието е вярно.

`while (condition) { statements..... }`

condition – булев израз. Цикълът се изпълнява докато той е истина.

statements – оператори, които се изпълняват от цикъла.

3.2.2. Цикъл do

Цикъл подобен на while, но с тази разлика, че условието се проверява в края на цикъла.

`do { statements..... }`

`while (condition);`

statements – оператори, които се изпълняват от цикъла.

condition – булев израз. Цикълът се изпълнява, докато изразът е истина.

4. Оператор break и continue

Операторите break и continue променят начина, по който работи цикълът.

break – кара кода да излезе от цикъла, без да изпълнява повече оператори във вътрешността на цикъла.

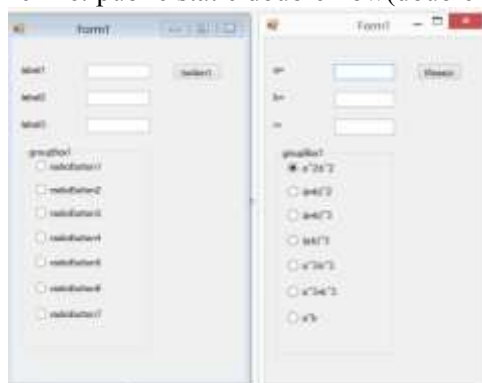
continue – оператор позволява прекратяването на текущата итерация на цикъл и започването на нова итерация. Пропускат се останалите оператори в блока от оператори [1], [10]

5. Задачи за изпълнение

Задача 1. Да се създаде приложение Математически

формули включващо показателно уравнение от вида a^b , както и формули за съкратено умножение $(a+b)^2$, $(a-b)^2$, $(a+b)*(a-b)$, $(a+b)^3$, $(a-b)^3$, a^3-b^3 и a^3+b^3 . Да се извърши проверка за коректни входни данни. Пресмятането да е автоматично непосредствено след въвеждане на стойности за a и b и избор на формула за прилагане.

Забележка: Методът Math.Pow() в C# се използва за изчисляване на число, повдигнато на степен на друго число. Синтаксисът е: `public static double Pow(double val1, double val2);`



Частично решение:

```
double a, b, Result;
a = Convert.ToDouble(textBox1.Text);
b = Convert.ToDouble(textBox2.Text);
if (radioButton1.Checked) {
    Result = Math.Pow((a + b), 2);
    textBox3.Text = Result.ToString(); }
if (radioButton7.Checked) {
    Result = Math.Pow(a, 3) + Math.Pow(b, 3);
    textBox3.Text = Result.ToString(); }
```

Задача 2. Да се създаде приложение Калкулатор за дължими осигуровки. Да се ограничат допустимите символи при въвеждане, така, че да включват цифри от 0 до 9, запетая и Backspace. Резултатът да се закръгли до втори знак след десетичната запетая, да е форматиран съобразно мерна единица за валута – лев. Пресмятането да е автоматично и да се извършва непосредствено след въвеждане на коректни входни данни.

Примерно решение:

...using System.Globalization;

```
namespace UP3_1 {
private void Form1_Load(object sender, EventArgs e) {
    label1.Text = "Salary";
    // DDO - Income tax
    label2.Text = "DOO";
    // DZPO - The additional obligatory pension insurance
    label4.Text = "DZPO";
    label6.Text = "Health Insurance";
    label8.Text = "Total";
    radioButton1.Text = "Before 1960 year";
    radioButton1.Checked = true;
    radioButton2.Text = "After 01.01.1960 year";
    label3.ResetText();
    label5.ResetText();
    label7.ResetText();
    label9.ResetText();
    groupBox1.Text = "Year of birth";
    button1.Text = "Calculate"; }
string BGL="";

void ShowAmmount(double aAmmount, CultureInfo aCulture) {
    BGL = String.Format(aCulture, "{0:C}", aAmmount); }

private void button1_Click(object sender, EventArgs e) {
    double pDDO, pDZPO, Salary, DOO, DZPO, Total, HI;
    if (radioButton1.Checked) {
        pDDO = 0.089; pDZPO = 0;
        Salary = Convert.ToDouble(textBox1.Text);
        DOO = pDDO * Salary;
        ShowAmmount(DOO, new CultureInfo("bg-BG"));
        label3.Text = BGL;
        DZPO = pDZPO * Salary;
        ShowAmmount(DZPO, new CultureInfo("bg-BG"));
        label5.Text = BGL;
        HI = 0.032 * Salary;
        ShowAmmount(HI, new CultureInfo("bg-BG"));
        label7.Text = BGL;
        Total = DOO + DZPO + HI;
        ShowAmmount(Total, new CultureInfo("bg-BG"));
        label9.Text = BGL; }
    if (radioButton2.Checked) {
        pDDO = 0.067; pDZPO = 0.022;
        Salary = Convert.ToDouble(textBox1.Text);
        DOO = pDDO * Salary;
        ShowAmmount(DOO, new CultureInfo("bg-BG"));
        label3.Text = BGL;
        DZPO = pDZPO * Salary;
        ShowAmmount(DZPO, new CultureInfo("bg-BG"));
        label5.Text = BGL;
        HI = 0.032 * Salary;
        ShowAmmount(HI, new CultureInfo("bg-BG"));
        label7.Text = BGL;
        Total = DOO + DZPO + HI;
        ShowAmmount(Total, new CultureInfo("bg-BG"));
        label9.Text = BGL; } }

private void textBox1_KeyPress(object sender, KeyPressEventArgs e) {
    //allow comma, BackSpace and Digit from 0 to 9
    if (!Char.IsDigit(e.KeyChar) && e.KeyChar != (Char)Keys.Back
```

```
&& e.KeyChar != '.'){
e.Handled = true;
textBox1.Focus();}}}
```

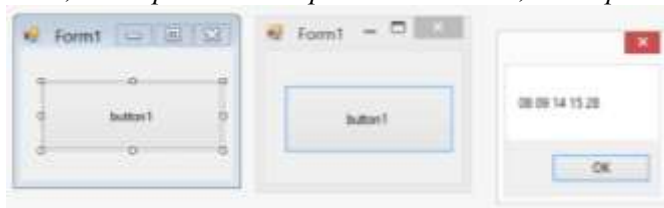
Задача 3. Да се създаде приложение Календар. При задаване на конкретна дата в *dateTimePicker*, да се извежда следната информация: ден, месец и година; ден от седмицата (Понеделник, Вторник и т.н.); брой дни от посочения месец; поредният ден от годината; определя се дали годината е високосна или не; брой дни до нова година; определя се разликата в дни, месеци, години спрямо текущата дата. **Приложението да се допълни с масив от всички официални празници от календара на Република България и при избор на дата да се извърши проверка за съответствие със списъка, като се изведе подходящо съобщение.**



Частично решение:

```
private void button1_Click(object sender, EventArgs e) {
    DateTime dd = new DateTime();
    dd = DateTime.Parse(dateTimePicker1.Text);
    string Message = "\nDay=" + dd.Day + "\nMonth=" + dd.Month +
        "\nYear=" + dd.Year + "\nDayOfWeek=" + dd.DayOfWeek +
        "\nDayOfYear=" + dd.DayOfYear;
    string Message1 = "30 Day";
    string Message2 = "31 Day";
    string Message3 = "28 Day";
    string Message4 = "29 Day";
    int mm = dd.Month;
    switch (mm) {
        case 1: ...
        case 12: MessageBox.Show(Message+"\n"+Message2); break;
        case 4: ...
        case 11: MessageBox.Show(Message+"\n"+Message1); break;
        case 2: if (DateTime.IsLeapYear(dd.Year)){
            MessageBox.Show("Leap Year! " + Message + "\n" + Message4); }
            else {
                MessageBox.Show("Not a Leap Year! " + Message + "\n" + Message3); } break; } }
}
```

Задача 4. Да се създаде приложение Спортен тотализатор, като се генерират случайни числа в интервала от 1 до 49, 6 на брой и в интервала от 1 до 35, 5 на брой.



Примерно решение:

```
using System;
namespace Loterry { public static Random random;
public static List<int> lottoNumbers = Enumerable.Range(1, 35).ToList();
private void button1_Click(object sender, EventArgs e){
    random = new Random((int)DateTime.Now.Ticks);
    var LinesToGenerate = 1; GenerateNumbers(LinesToGenerate);}
public static void GenerateNumbers(int LineCount){
    int[] SelectedNumbers = new int[5];
    for (var i = 0; i < 5; i++){
        var number = GetRandomNumber(lottoNumbers.ToArray());
        while (SelectedNumbers.Contains(number))
            number = GetRandomNumber(lottoNumbers.ToArray());
    }
}
```

```

        SelectedNumbers[i] = number;}
        var numbersOrdered = SelectedNumbers.OrderBy(n => n).Select(n =>
'0'));
        MessageBox.Show(string.Join(" ", numbersOrdered));
        if (LineCount > 1) GenerateNumbers(--LineCount);}
public static int GetRandomNumber(int[] arr) {
    if (arr.Length > 1){
        var r = random.Next(0, arr.Length);
        var list = arr.ToList();
        list.RemoveAt(r);
        return GetRandomNumber(list.ToArray());}
    return arr[0]; } } }

```

Забележка: В примерното решение се използват extension методи с ламбда изрази. Ламбда изразите са анонимни функции, които съдържат изрази или последователност от оператори. Всички ламбда изрази използват ламбда оператора $\Rightarrow$, който може да се чете като "отива в". Лявата страна на ламбда оператора определя входните параметри на анонимната функция, а дясната страна представлява израз или последователност от оператори, която работи с входните параметри и евентуално връща някакъв резултат. В решението са използвани и разширяващи методи, като `OrderBy()`, `Select()`, `PadLeft()`. Разширяващите методи (extension methods) се дефинират като статични методи в обикновени статични класове. Типа на първият им аргумент представлява класа (или интерфейса) към който се закачат. Преди него се слага ключовата дума `this`. За да могат да се достъпят дадените разширяващи методи, трябва да се добави с `using` съответното пространство от имена, в което е дефиниран статичния клас описващ тези методи. В противен случай, компилаторът няма как да разбере за тяхното съществуване.

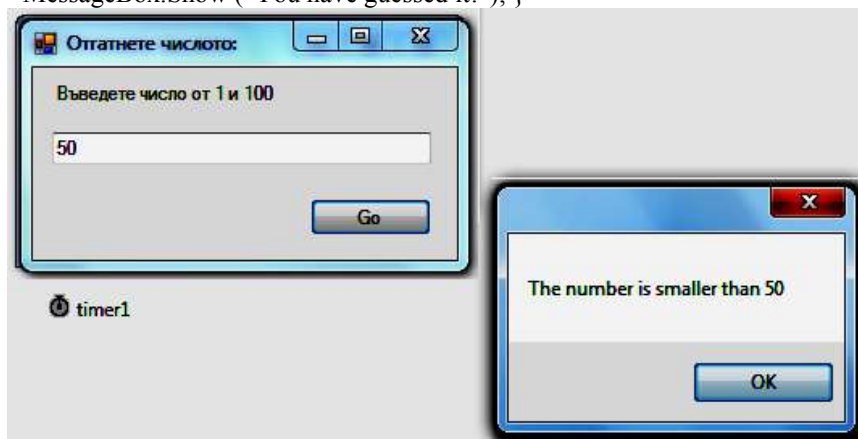
Задача 5. Да се напише игра Отгатни числото. Да се добави таймер, който отмерва времето до края на играта (фиг.3.1.)

Частично решение:

```

static Random random = new Random();
private int randomNumber = random.Next(99) + 1;
private void button1_Click(object sender, EventArgs e){
    textBox2.Text = randomNumber.ToString();
    if (int.Parse(textBox1.Text) < randomNumber)
        MessageBox.Show("It's greater than "+textBox1.Text+"!");
    if (int.Parse(textBox1.Text) > randomNumber)
        MessageBox.Show("It's less than " + textBox1.Text + "!");
    if (int.Parse(textBox1.Text) == randomNumber)
        MessageBox.Show("You have guessed it!"); }

```



Фиг.3.1. Интерфейс на приложение Отгатни числото, в режим изпълнение

6. Задачи за самостоятелна работа

Задача 1. Да се създаде приложение Лихвен калкулатор, което по зададена сума пресмята проста и сложна лихва за ден, месец и година.

Формули за проста лихва:

$$K_n = K_0(1 + (p \cdot n)/100) \quad (3.1.)$$

K_n – сума за погасяване K_0 – изтеглена сума

p – процент за олихвяване n – период за година

$n = m/12$, където m е период в месеци

$n = t/V$, където t е период в дни, $V=360$ дни (приема се, че 1 месец има средно 30 дни)

Формули за сложна лихва:

$$K_n = K_0(1 + p/100)^n \quad (3.2.)$$

Задача 2. *Да се създаде приложение Морски шах. Резултатът от играта да се съхранява в текстов файл, с добавяне на записи след всяка нова игра.*

Забележка: Работа с текстов файл подробно се разглежда в тема V.

Задача 3. *Да се създаде приложение Рождена дата, което по зададен ден, месец и година, определя възрастта на потребителя.*

Задача 4. *Като се използва Class Library да се създаде dll файл, който реализира функция за събиране на две реални числа. Да се извика в приложение Калкулатор.*

Забележка: DLL е съкращение от Dinamic Link Library (библиотека за динамично свързване). Това са библиотечни файлове с компилиран код, който може да се употребява в други приложни програми. За създаването на dll в C# се избира Class Library, а за извикването от падащо меню References/Add Reference се избира създадения dll файл.